

Sol Cheatsheet

Sol Cheatsheet

Quick reference for ASU's Sol supercomputer — SLURM basics, the `solx` CLI and its raw-SLURM equivalents, partitions & QOS, Sol's own wrappers, and getting at a compute-node service from your laptop.

A rendered PDF lives at [docs/cheatsheet.pdf](#) (build it with `scripts/build-cheatsheet.sh`). In a terminal on Sol, run `solx cheatsheet` to print this page.

Know your access first

What partitions, QOS, and group account *you* can use — the answer drives every job decision below:

```
sacctmgr -n show assoc user=$USER format=Account,Partition,QOS
# -> e.g.  grp_yourpi || debug,htc,private,public
sshare -U          # your fairshare (lower = back off / use a buy-in QOS)
```

Partitions — pick by wall-time, not by “is it a GPU job?”

GPUs live in `htc`, `public`, and `general`. The deciding question is *how long* and *how urgently*, not CPU-vs-GPU.

Partition	Wall limit	GPUs	Use it for
<code>htc</code>	4 h	large A100 pool + H100/L40/A30/H200	the default for anything ≤ 4 h, GPU included — least
<code>public</code>	7 days	A100 (+ A100-MIG, A30)	contended runs that need > 4 h,
<code>general</code>	14 days	A100/H100/H200/L40	non-preemptable privately-owned nodes (via <code>-q private</code> or your <code>grp_*</code>)
<code>lightwork</code>	1 day	a100.20gb	the <code>vscode</code> tunnel's home;
<code>highmem</code>	7 days	—	light dev up to 2 TB RAM

QOS — priority & preemption, and which partitions accept it

QOS	Wall cap	Notes
<code>public</code>	(partition's)	default, non-preemptable

QOS	Wall cap	Notes
debug	15 min	very high priority; GPUs OK; public/general only — rejected on htc ;
private	(partition's)	one job at a time preemptible access to buy-in nodes — owners can cancel you;
grp_*	up to 30 days	runs past htc's 4 h your group's owned nodes (if you're in one)
class	1 day	course users; GPU-minute caps

Routing in one line: `<=4 h (incl. GPU) -> htc · <=15 min & urgent -> -p public -q debug · >4 h -> -p public · >4 h preemptible -> -p general -q private`. Never `-p htc -q debug` (invalid).

SLURM basics

```

sbatch job.sh          # submit a batch script
squeue --me            # your jobs          (alias: myjobs; bare `sq` = whole cluster)
scancel <jobid>        # cancel
scontrol show job <jobid> # full detail / why pending
sbatch --test-only job.sh # validate partition/QOS/time/gres WITHOUT submitting
interactive            # quick shell; defaults to -p htc -q public -c 1 -t 0-4

```

#SBATCH header skeleton (time format is D-HH:MM:SS):

```

#!/bin/bash
#SBATCH -p htc          # partition (htc = <=4h, has GPUs)
#SBATCH -q public       # QOS
#SBATCH -t 0-04:00:00   # wall-time
#SBATCH -c 8            # cores
#SBATCH --gres=gpu:a100:1 # GPU(s)
#SBATCH --mem=64G
#SBATCH -o slurm.%j.out

```

Start from Sol's templates, don't hand-roll headers: `/packages/public/sol-sbatch-templates/templates/`.

solx <-> raw SLURM

solx owns the interactive-allocation lifecycle; raw SLURM is the equivalent fallback for one-off reads.

solx	raw SLURM equivalent
<code>solx job start [TEMPLATE]</code>	<code>salloc / interactive</code> from a config template, <i>waits for the grant</i>
<code>solx job jump</code>	<code>srtn --pty \$SHELL</code> onto the compute node
<code>solx job list</code>	<code>squeue --me</code>
<code>solx job time</code>	<code>squeue -h -j "\$SLURM_JOB_ID" -o %L</code>
<code>solx job stop</code>	<code>scancel <jobid></code>
<code>solx keep</code>	renew the mtime on <code>/scratch</code> files Sol flagged (filtered by <code>[keep]</code>)
<code>solx job start gpu -- ...</code>	anything after <code>--</code> is appended to <code>salloc</code> (last flag wins)

Config lives at `~/.config/solx/config.toml` (`solx config edit`). Add `--json` for machine output; `-n` to preview; `-y` to skip prompts.

Sol's own `my*` / `show*` wrappers

You want	Command
Your fairshare / priority	<code>myfairshare</code>
Your <code>/scratch</code> quota	<code>beegfs-ctl --getquota --uid \$USER</code>
Your jobs right now	<code>myjobs</code> (or <code>squeue --me</code>)
Estimated start of a pending job	<code>thisjob <jobid></code>
Efficiency of a finished job	<code>seff <jobid></code>
Free capacity / partitions	<code>sinfo, showparts</code>
Free GPUs per node	<code>showgpus</code>

Reaching a compute-node service from your laptop

VS Code: from a Sol login node, register a tunnel (wraps `srunc` on `lightwork`)
`vscode` # then open the tunnel named `sol_$USER`

Manual port-forward (e.g. Jupyter on `$NODE:8888`), run from your LAPTOP:
`ssh -N -L 8888:localhost:8888 -J $USER@login.sol.rc.asu.edu $USER@$NODE`

`$NODE` is the compute node your allocation landed on (`squeue --me -> NODELIST`). Bind services to `localhost`, never `0.0.0.0`, on shared nodes.

Storage & caches

Path	For	Lifetime
<code>/scratch/\$USER</code>	datasets, model caches, run outputs	purged after inactivity — renew with <code>solx keep</code>
<code>/home/\$USER</code>	code, configs, <code>~/.local</code> installs	persistent, small quota

Point heavyweight caches at `/scratch`, not `/home`:

```
export HF_HOME=/scratch/$USER/.cache/huggingface
export UV_CACHE_DIR=/scratch/$USER/.cache/uv
```

Heavy I/O — where to run it

Login nodes are throttled. For a big metadata pass (e.g. touching hundreds of thousands of files) use the **DTN** (`ssh soldtn`), a **compute node** (interactive), or a short **htc batch job** — never the login node.